

I2236 INTRODUCTION TO EMBEDDED SYSTEMS

LECTURE 3 **DIGITAL & ANALOG INTERFACING**

Bilal Daass
04/02/2026



COURSE OVERVIEW

Module 1: Introduction & Foundations (Week 1–2)

Module 2: Arduino Ecosystem & Programming Basics (Week 3–4)

Module 3: Digital & Analog Interfacing (Week 5–6)

Module 4: Interfacing Peripherals (Week 7–8)

Module 5: Sensors & Actuators (Week 9–10)

Module 6: Communication & Integration (Week 11–12)

Module 7: Project (Week 13–15)

MODULE 3 : DIGITAL & ANALOG INTERFACING

- **Digital I/O: LEDS, switches, relays, transistors**
- **PWM: theory and applications (LED dimming, RGB LED, motor control)**
- **ADC: reading analog signals (potentiometer, LDR, temperature sensor)**



SWITCHES

SWITCHES

Switch is used to turn on/off devices (e.g. room light).

When a switch is open, it interrupts the flow of electricity.

When a switch is closed, it will complete a circuit.

Pushbutton	Toggle button	Slide switch	Reed switch	Tilt/ball switch
				
			 	

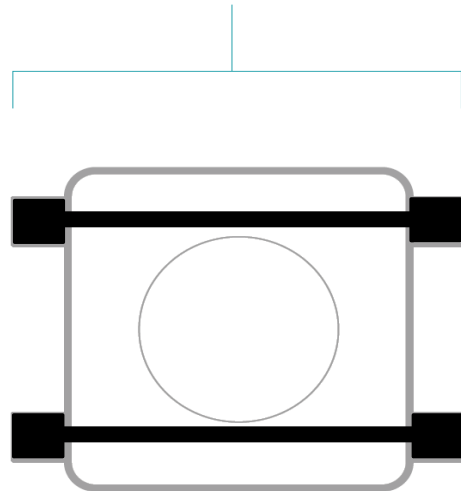
SWITCHES

Pushbutton

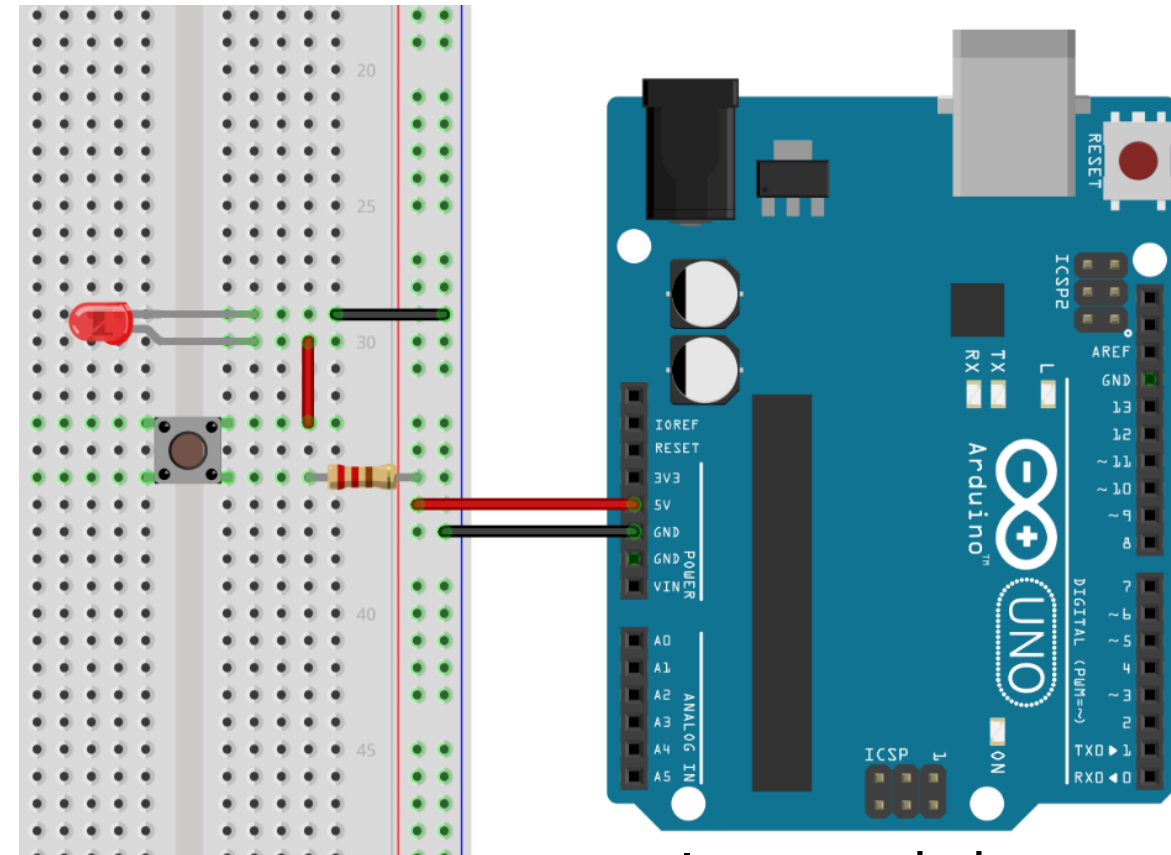


These two pins of a switch are connected to each other

These two are not.
They form the switch



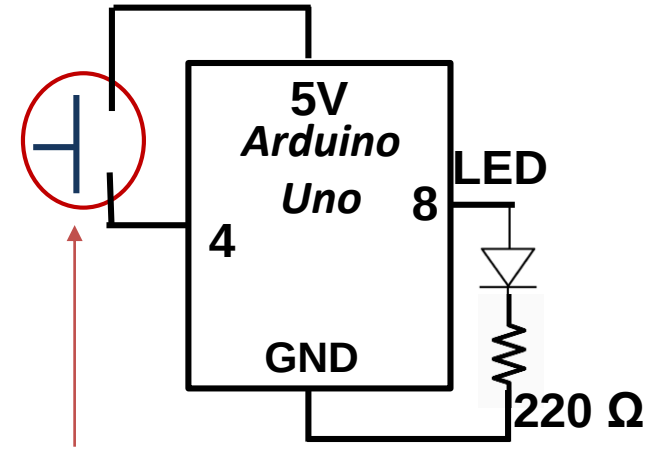
Make a circuit to turn the LED ON or OFF when the switch is continuously pressed.



Input: switch
Output: LED

SWITCHES

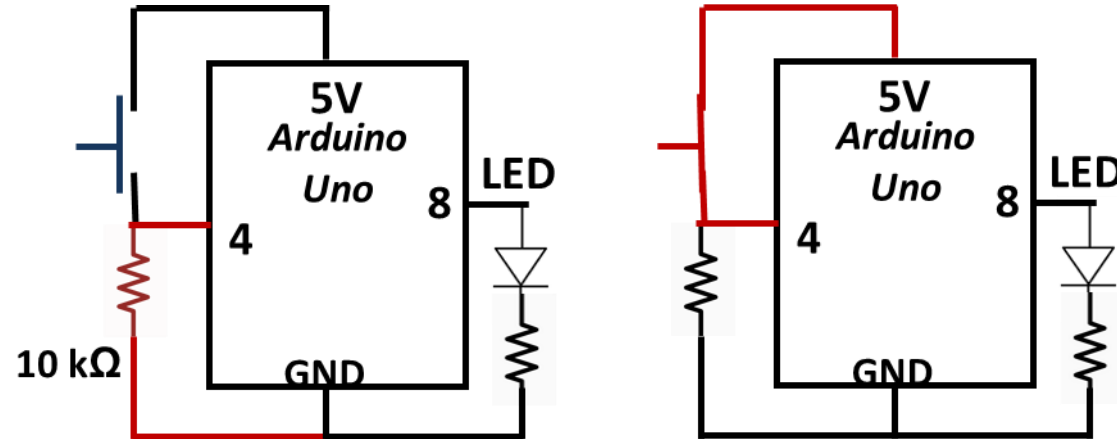
Switch with pull-down/pull-up resistor



*Pin is floating (on the air)
between HIGH/LOW due to noise*

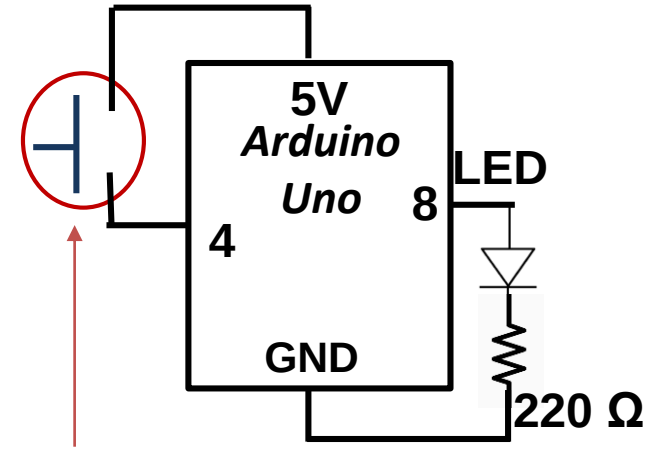
Solution 1

Add a **Pull-down** resistor (resistor connected to **ground**) to keep the PIN STATE to **LOW** when the switch is **opened**.



SWITCHES

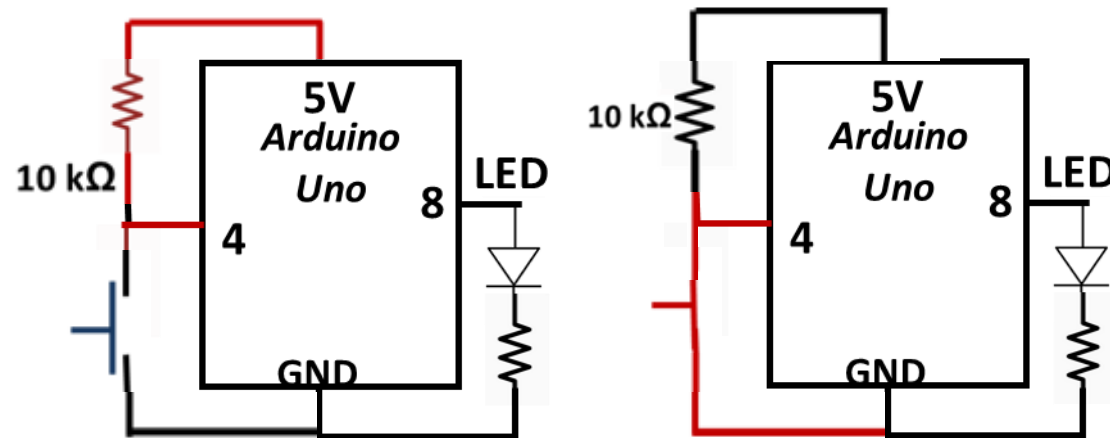
Switch with pull-down/pull-up resistor



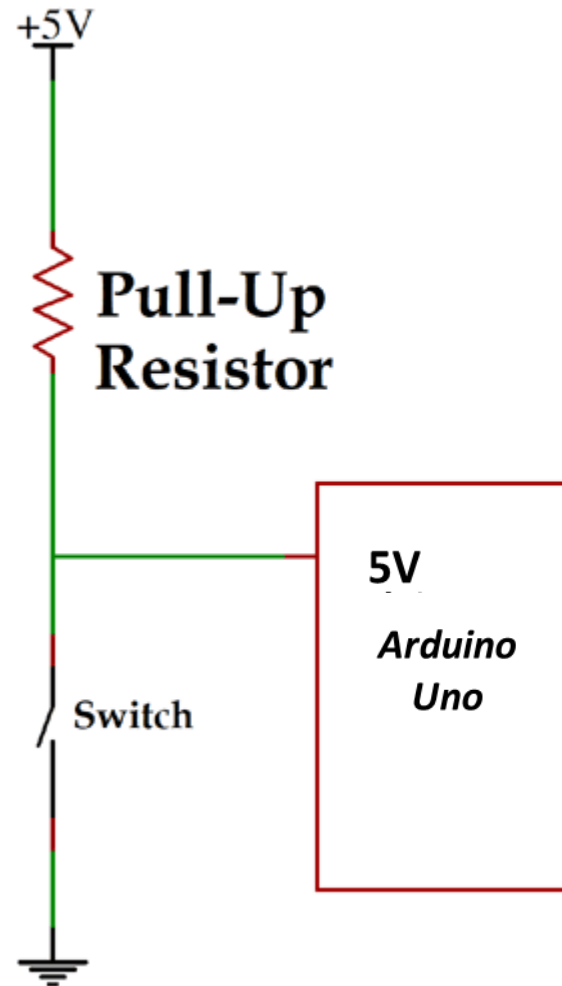
*Pin is floating (on the air)
between HIGH/LOW due to noise*

Solution 2

Add a **Pull-Up** resistor (resistor is connected to 5V) : When the switch is open : The pin state is HIGH (=5V)

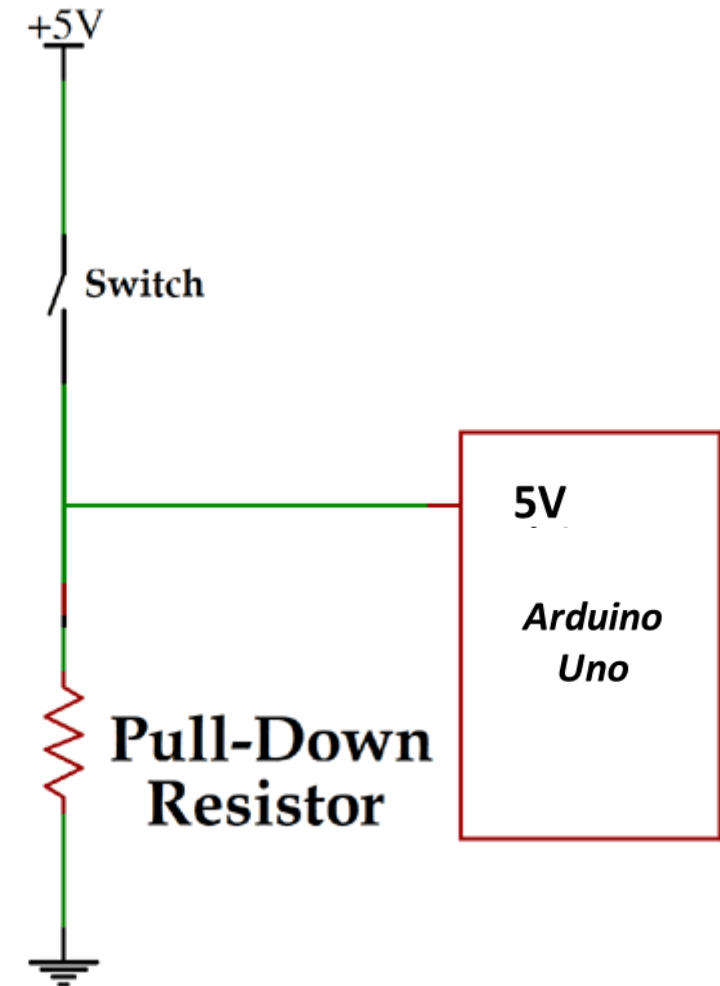


SWITCHES



Pull-Up Resistor

When the switch is open
The pin state is HIGH (=5V)



Pull-Down Resistor

When the switch is open
The pin state is LOW (=0V)

SWITCHES

EXERCICE :

1. Turn the LED on when the push button is continuously pressed and OFF when the push button is released.
2. Modify the solution for Question 1 to achieve the same result using an if-else statement
3. Develop an Arduino project that contains 2 LEDs and 1 pushbutton
 - When the pushbutton is pressed, the first LED is turned on and the second LED is turned off
 - When the pushbutton is released, the first LED is turned off and the second LED is turned on

SWITCHES

Exercise 1 - Solution

```
int switchPin = 4; // define switch pin
```

```
int ledPin = 8; // define LED pin
```

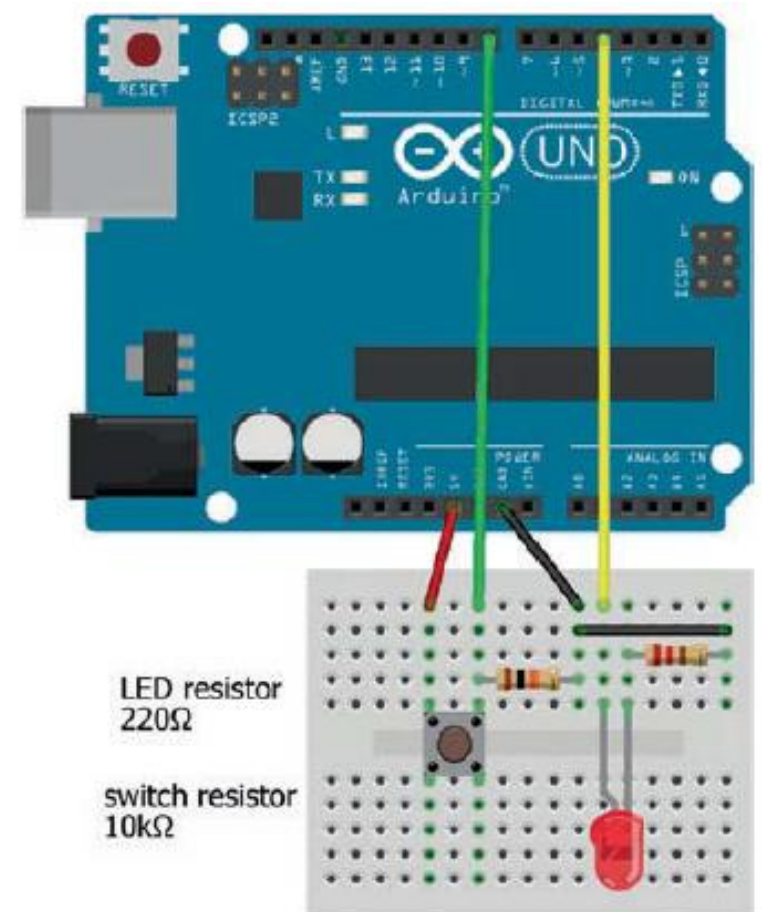
```
int reading;
```

```
void setup()
```

```
{  
  pinMode(switchPin, INPUT); // switch pin as input  
  pinMode(ledPin, OUTPUT); // LED pin as output  
}
```

```
void loop()
```

```
{  
  reading = digitalRead(switchPin); // read switch pin  
  digitalWrite(ledPin, reading); // the value of reading is HIGH or LOW, LED will be OFF if reading is LOW  
}
```



SWITCHES

Exercise 1 – Solution – with if statement

```
int switchPin = 8; // define switch pin
int ledPin = 4; // define LED pin
int reading;
void setup()
{
    pinMode(switchPin,INPUT); // switch pin as
input
    pinMode(ledPin,OUTPUT); // LED pin as
output
}
```

```
void loop()
{
    reading = digitalRead(switchPin); // read switch pin
    if(reading == HIGH) // check if reading is HIGH
//if yes then turn the LED on, otherwise turn the LED off
    {
        // the pushbutton is pressed
        digitalWrite(ledPin,HIGH); // turn the LED on
    }
    else
    {
        digitalWrite(ledPin,LOW); // turn the LED off
    }
}
```



PWM-RGB LED

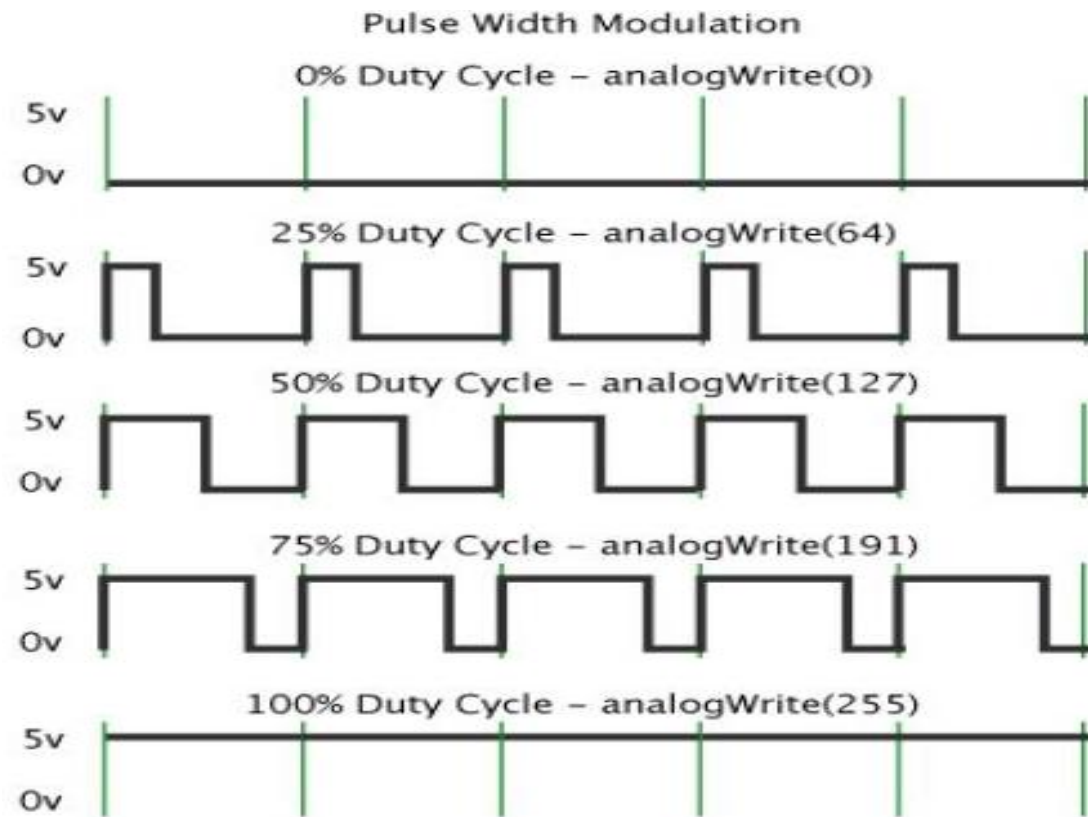
PWM

- ❑ **Pulse Width Modulation (PWM)** is a technique used to simulate **analog results** using **digital signals**.
- ❑ Digital controllers create a square wave—a signal that rapidly switches between two states: ON (5v) and off (0V).



- ❑ The key to PWM is managing the timing of these states. The duration the signal remains in the **ON** state during a single cycle is known as the **Pulse Width**.
- ❑ To achieve different "analog" levels, you **modulate** (change) that pulse width.
A longer pulse width delivers more average power.
A shorter pulse width delivers less average power.
- ❑ If you repeat this on-off pattern fast enough with an LED for example, the result is as if the signal is a steady voltage between 0 and 5v controlling the brightness of the LED.

PWM



- 0% (0/255) $\rightarrow$ 0 V
- 25% (64/255) $\rightarrow$ 1.25 V
- 100% (255/255) $\rightarrow$ 5 V

PWM

- PWM is generated by `analogWrite(pin, value)` where “value” is between 0 and 255
- The PWM pins are labeled on the board with tildes (~) next to the pin number. (Pins 3, 5, 6, 9, 10, 11).

Duty cycle = $\text{value}/255 * 100$



PWM - EXERCICE

Fading LED

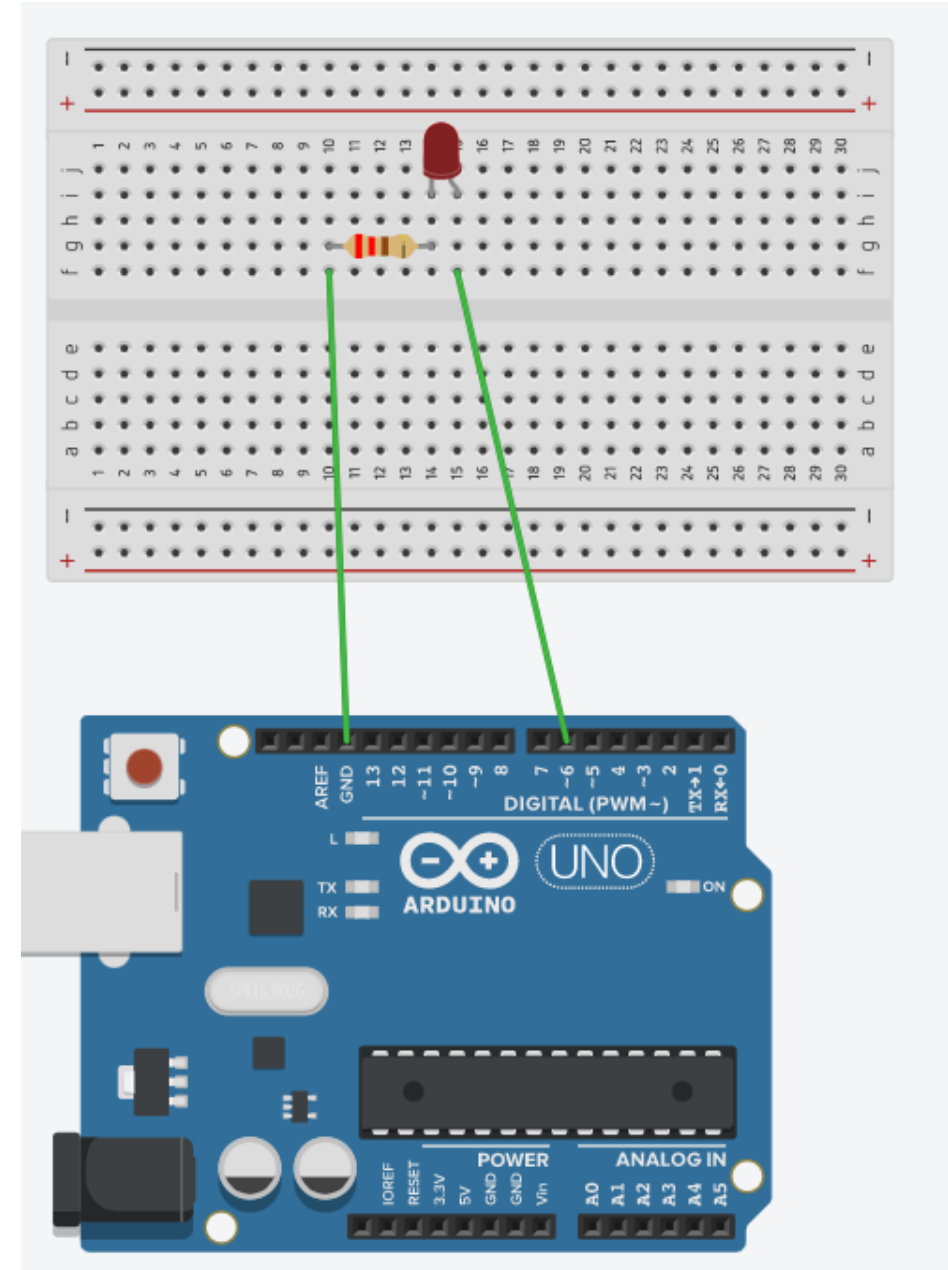
Connect an LED to one of the PWM pins and compose a simple program to slowly fade the LED dimmer.

PWM - EXERCICE

Fading LED - Solution

```
int ledPin = 6;  
int delayTime = 1000;  
int brightness;  
  
void setup()  
{  
  pinMode(ledPin, OUTPUT);  
}
```

```
void loop()  
{  
  brightness = 255;  
  analogWrite(ledPin, brightness);  
  delay(delayTime);  
  
  brightness = 127;  
  analogWrite(ledPin, brightness);  
  delay(delayTime);  
  
  brightness = 64;  
  analogWrite(ledPin, brightness);  
  delay(delayTime);  
  
  brightness = 0;  
  analogWrite(ledPin, brightness);  
  delay(delayTime);  
}
```



RED GREEN BLUE LED (RGB)

An RGB LED consists of three combined LEDs:

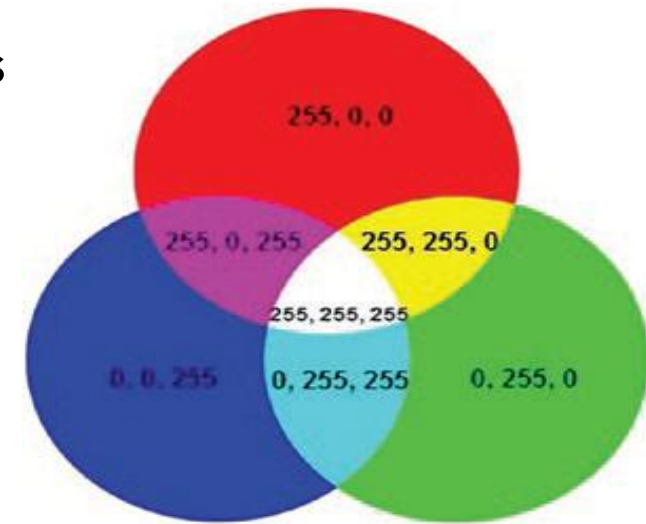
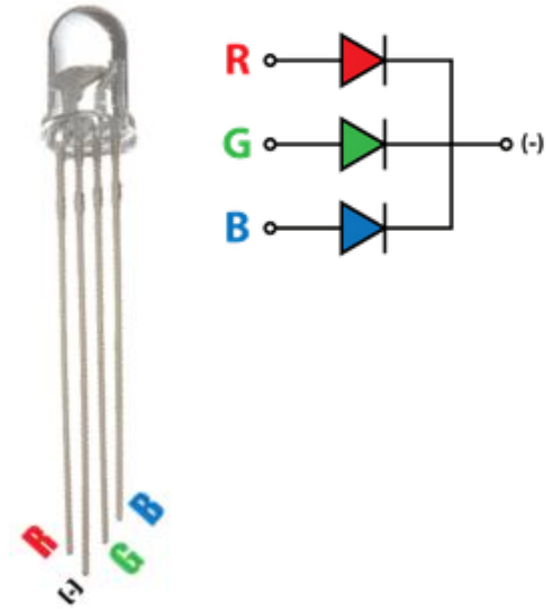
A red, a green, and a blue LED.

The RGB LED has a common cathode which has the longest leg.

The legs R, G and B should be connected to the PWM pins

An RGB LED is activated with `analogWrite(pin, value)` where the *pin* is the PWM pin number for the red, green, or blue LEDs and the *value* is the light intensity value, it is represented by an integer between 0 and 255.

A 220Ω resistor is connected between each of the RGB LEDs and the Arduino PWM pins.



A color can be defined as a combination of red, green, and blue components.

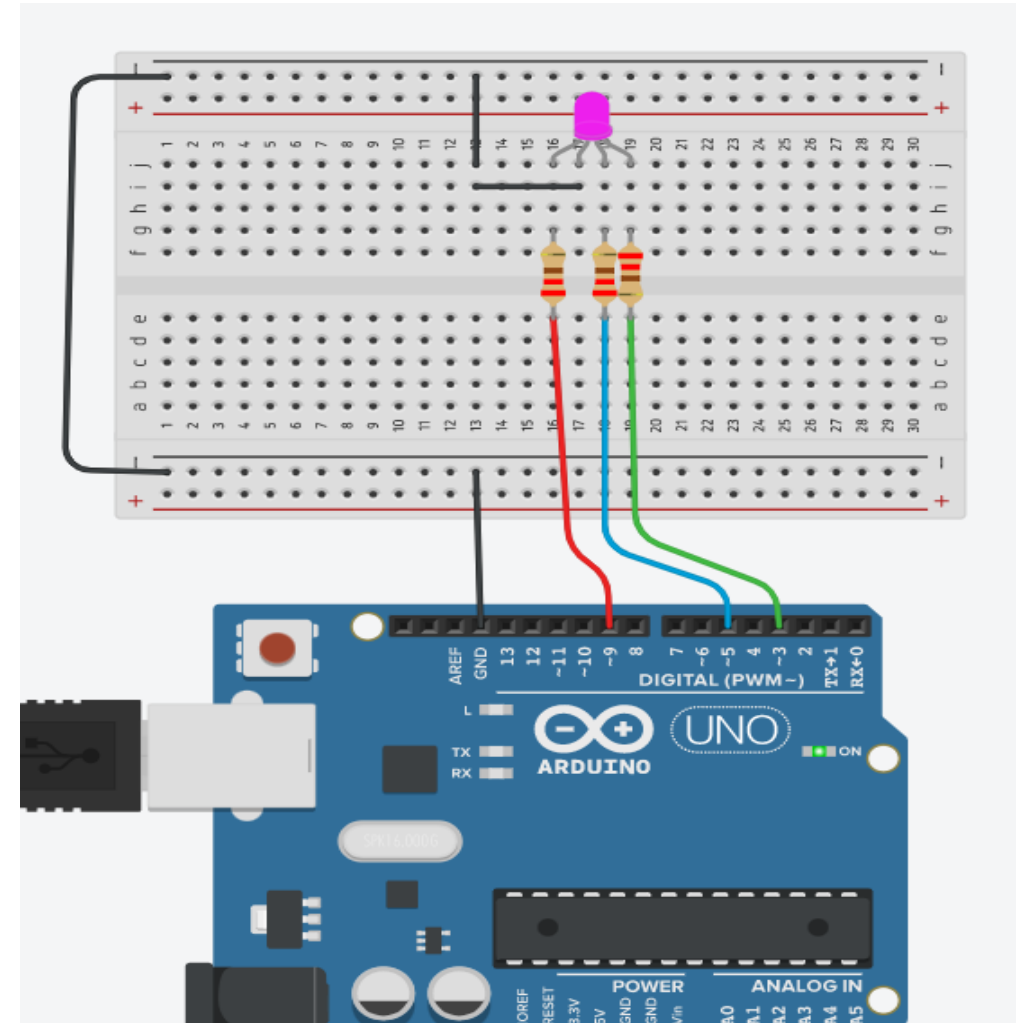
Visit https://www.w3schools.com/colors/colors_rgb.asp to check the possible combinations of the red, green, and blue colors.

EXAMPLE

```
int red = 9 ;
int green = 3;
int blue = 5;
void setup()
{
  pinMode(red, OUTPUT);
  pinMode(green, OUTPUT);
  pinMode(blue, OUTPUT);
}

void loop()
{
  analogWrite(red, 255);
  analogWrite(green, 0);
  analogWrite(blue, 255);
  delay(1000); // Wait for 1000 millisecond(s)

  analogWrite(red, 255);
  analogWrite(green, 255);
  analogWrite(blue, 0);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

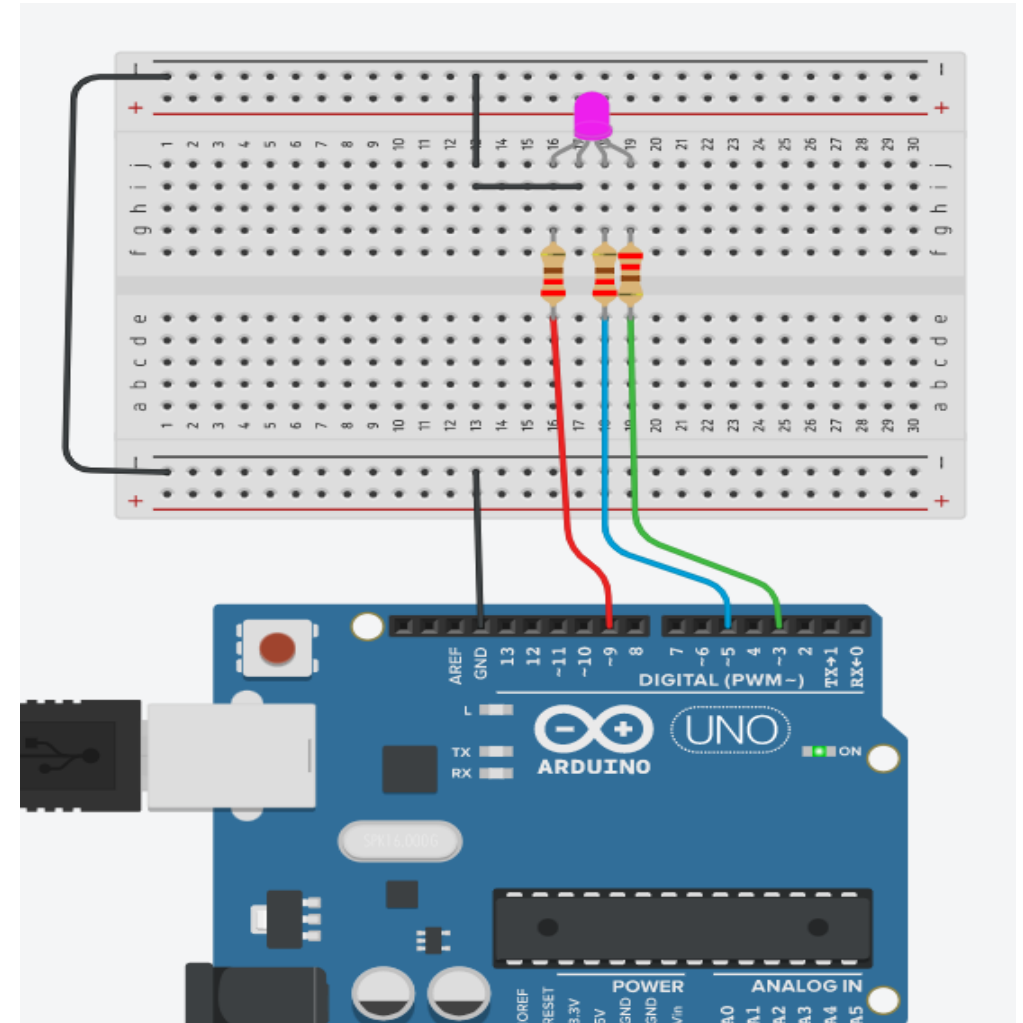


EXAMPLE 1

```
int red = 9 ;
int green = 3;
int blue = 5;
void setup()
{
  pinMode(red, OUTPUT);
  pinMode(green, OUTPUT);
  pinMode(blue, OUTPUT);
}

void loop()
{
  analogWrite(red, 255);
  analogWrite(green, 0);
  analogWrite(blue, 255);
  delay(1000); // Wait for 1000 millisecond(s)

  analogWrite(red, 255);
  analogWrite(green, 255);
  analogWrite(blue, 0);
  delay(1000); // Wait for 1000 millisecond(s)
}
```



EXAMPLE 2

DESIGN A POLICE CAR FLASH LIGHT:

- It uses 4 RGB LEDs:
 - Two outer crimson (220, 20, 60)
 - Two inner liberty (84, 92, 168)
- When the outer lights are flashing, the inner lights are not
- Then vice versa
- Each color stays bright for 0.2 seconds

SERIAL MONITOR

-

ADC

-

POTENTIOMETER

SENSORS

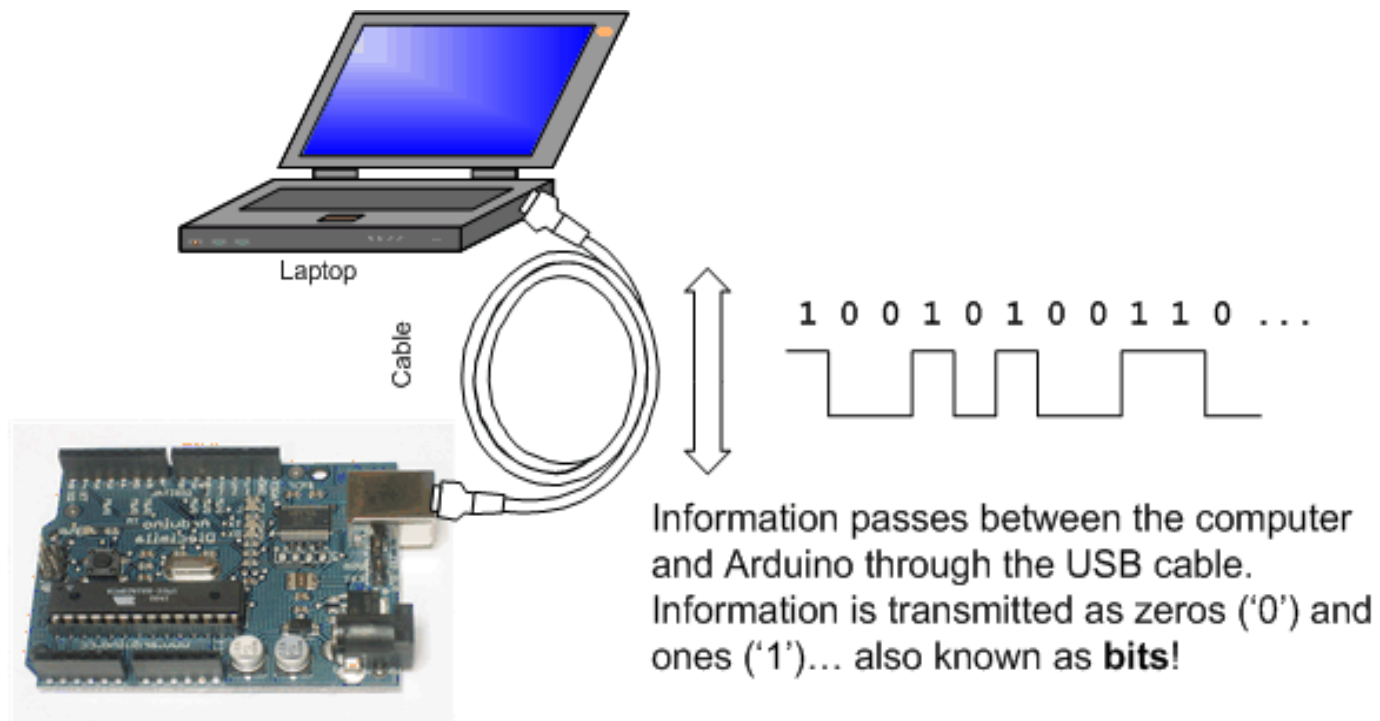
- Sensors are devices that convert a physical quantity, such as light intensity or temperature, into an electrical quantity.
- There are many different types of sensors:
 - ✓ Light sensor,
 - ✓ Motion sensor,
 - ✓ Temperature sensor,
 - ✓ Magnetic fields sensor,
 - ✓ Humidity sensor,
 - ✓ moisture sensor,
 - ✓ Vibration sensor,
 - ✓ Pressure sensor,
 - ✓ Electrical fields sensor,
 - ✓ Sound sensor,
 - ✓ Position sensor
 - ✓ ...
- Sensor information could displayed on the serial monitor or on the serial plotter.

SERIAL MONITOR

What is Serial?

The word **serial** means "one after the other."

Serial data transfer is when we transfer data one **bit** at a time, one right after the other.

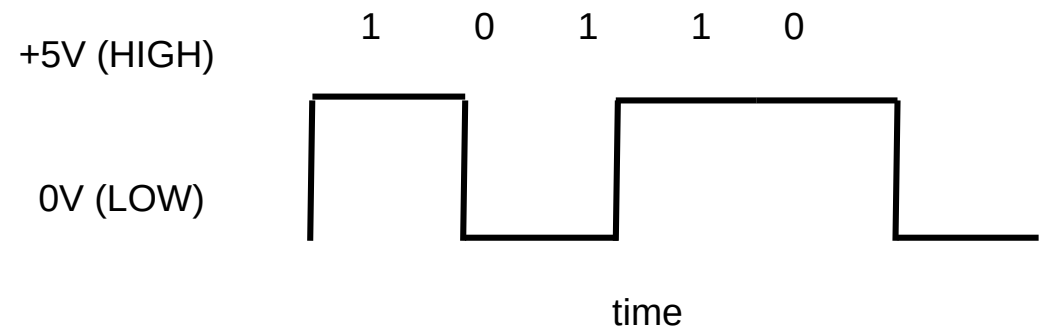
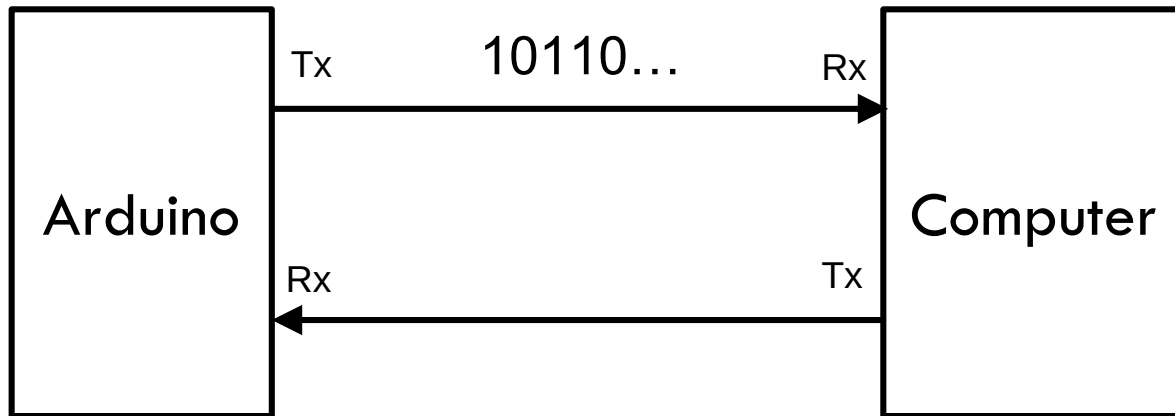


SERIAL MONITOR

Arduino Serial Interfaces

- Used for communication between the Arduino board and a computer or other devices.
- Arduino platforms have at least one serial port (also known as a UART or Universal Asynchronous Receiver/Transmitter)

Serial communication is via digital pins 0 (RX) & 1 (TX) and via USB to computer.



SERIAL MONITOR

SERIAL WRITE

- **Serial.begin():** Sets the data rate in bits per second (baud) for serial data transmission
- Syntax: `Serial.begin(baud)`
- Example: `Serial.begin(9600)` sets serial baud rate to 9600 bits per second
- **Serial.print():** Prints data to the serial port as human-readable ASCII text without carriage return / Newline Feed character
- Syntax: `Serial.print(val)` or `Serial.print(val, format)`
- Parameters:
 - val: the value to print - any data type
 - format: specifies the number base or number of decimal places
- Example: `Serial.print("Hello world.")` gives "Hello world."
- Example: `Serial.print(1.23456, 2)` gives "1.23"

SERIAL MONITOR

SERIAL WRITE

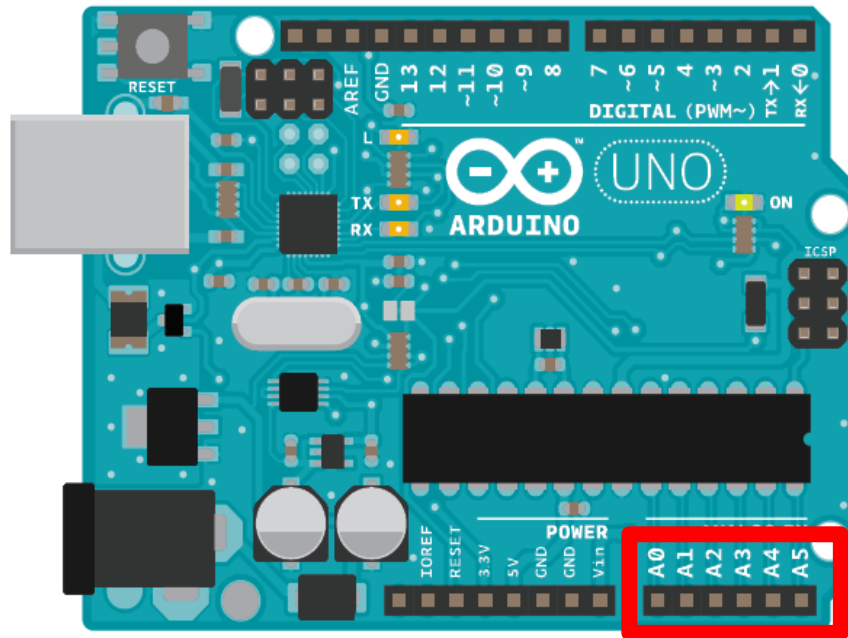
- **Serial.println():** Prints data to the serial port as human-readable ASCII text *followed by a carriage return and a newline character*
- Syntax: `Serial.println(val)` or `Serial.print(val, format)`
- Parameters:
 - val: the value to print - any data type
 - format: specifies the number base or number of decimal places
- Example: `Serial.println("Hello world.")` gives "Hello world."
- Example: `Serial.println(1.23456, 2)` gives "1.23"
- **Serial.write():** Writes binary data to the serial port. This data is sent as a byte or series of bytes
- Syntax: `Serial.write(val)`
- Example: `Serial.write("Hello world")` writes "Hello world"

SERIAL MONITOR - EXAMPLE

```
void setup()
{
  // open serial port at a baud rate of 9600 bits per second
  Serial.begin(9600);
}
void loop()
{
  Serial.println("Hello world!"); // prints to serial port
  Serial.println(1.23456, 2) // prints 1.23
  delay(1000);
}
```

ADC: READING ANALOG SIGNALS

- Arduino has 6 analog pins A0, ..., A5 that can read a voltage input.
- These pins are connected to the Arduino's Analog to Digital Converter (ADC)
- They convert an incoming analog signal **between 0V and 5V** into a range of numbers from **0-1023**.



Analog pins

- 10 bit resolution.
- 100 μ s (0.0001 s) to read an analog input.
- Reading rate = 10,000 /s

Analog signal can take any value between 0V and +5V
→ **analogRead(pinNumber)** instruction is used to read it

ADC: READING ANALOG SIGNALS

analogRead instruction reads the value from the specified analog pin

Example:

```
int val = 0;  
val = analogRead(A0); // read analog input at pin A0
```

val is an integer between 0 and 1023:

Input = 0V → val = 0

Input = +5V → val = 1023

For any voltage input **X** in-between, val is determined by applying the rule of three (cross-product rule):

$$val = \frac{X}{5} * 1023$$

Example, for input = 2.5V, val = (2.5/5)*1023 = 512

POTENTIOMETER

- A **potentiometer** is a three-terminal variable resistor.
- It can be used to control
 - light brightness of LED,
 - blink rate of LED,
 - motor speed,
 - sound volume,
 - etc...

A voltage divider can change an output voltage with a combination of resistors.

A voltage divider consists of an input voltage, **V_{in}** , two resistors, **R_1** and **R_2** , in series, and an output voltage, **V_{out}** , at the junction of the two resistors.

From Ohm's law, the output voltage $V_{out} = I \times R_2$,

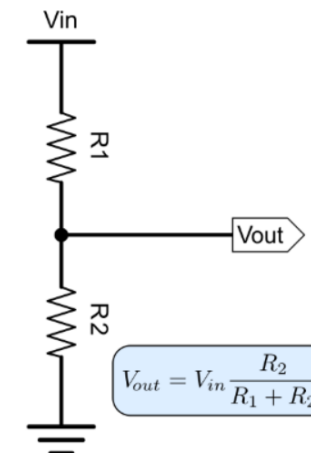
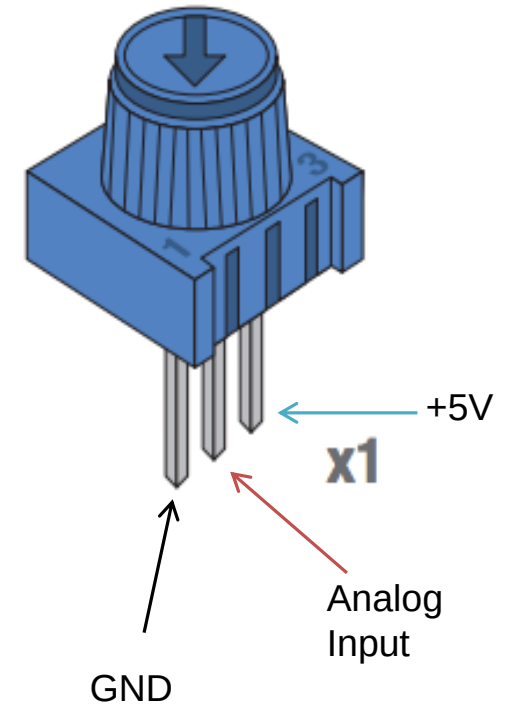
where I is the current through the circuit, equal to $\frac{V_{in}}{R_1 + R_2}$

so

$$V_{out} = V_{in} \left(\frac{R_2}{R_1 + R_2} \right)$$

If the two resistors are equal, then the output voltage is half the input voltage.

Connections

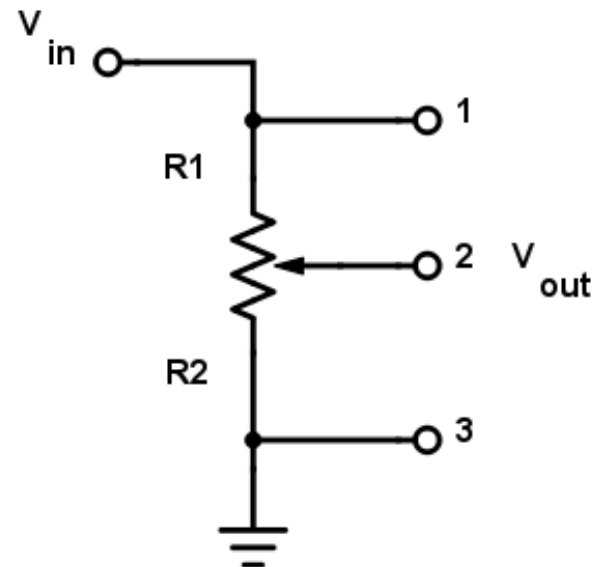
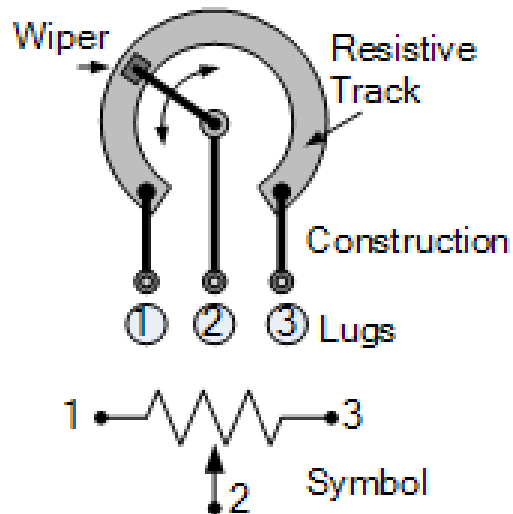


POTENTIOMETER

A **potentiometer** is an adjustable voltage divider

$$V_{out} = \frac{R_2}{R_1 + R_2} V_{in}$$

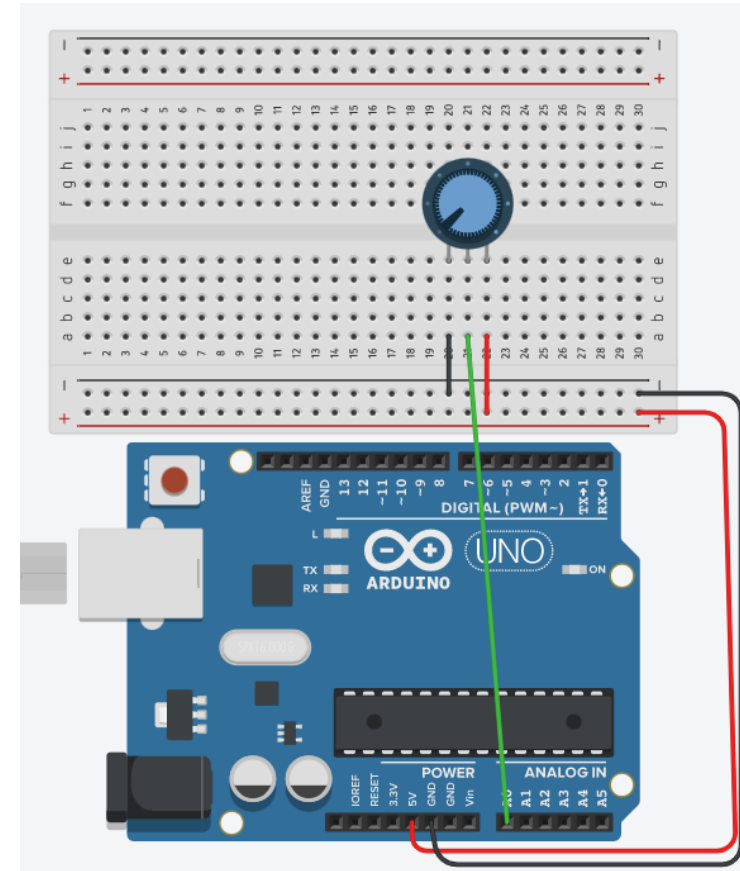
V_{out} varies between 0 and V_{in} volts by turning the wiper



POTENTIOMETER – EXAMPLE 1

Connect the potentiometer on the analog pin A0 and Read its value and display it on the Serial Monitor

```
int potPin = A0;
int val=0;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  val = analogRead(A0); // read the value on A0
  Serial.println(val); //display the value on the serial monitor
  delay(1000);
}
```

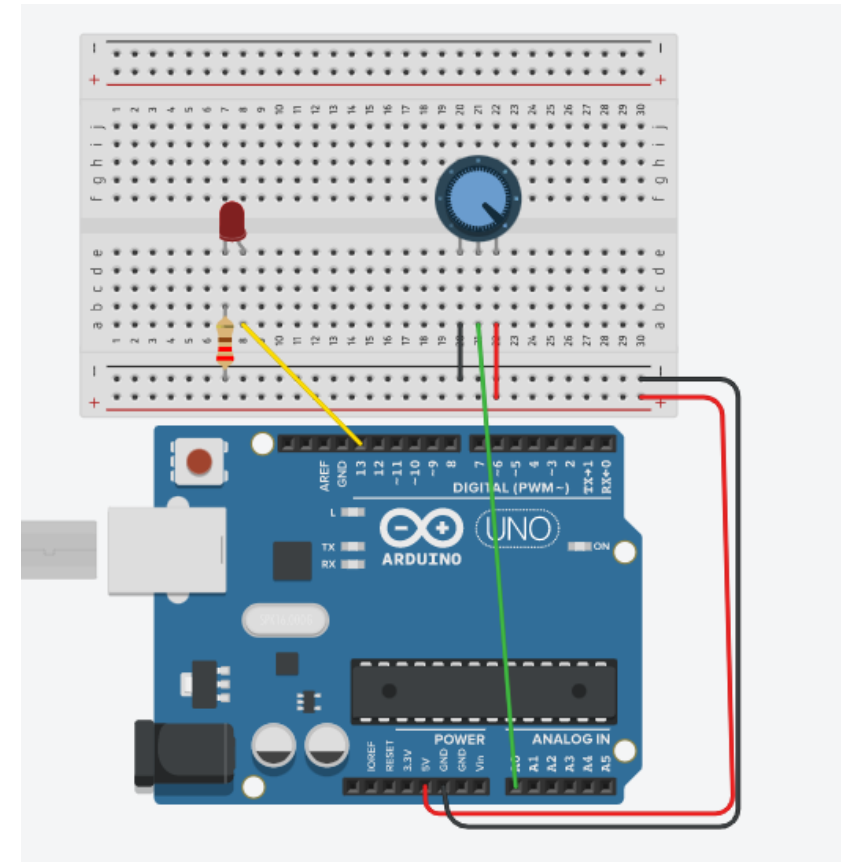


POTENTIOMETER — EXAMPLE 2

Write a code that controls the blink rate of the LED using potentiometer. The control is done by reading the value of the pot on analog pin A0 and turning on and off the LED connected to digital pin 13. The amount of time the LED will be ON and OFF depends on the value obtained by `analogRead()`.

Display the value obtained at A0 on the Serial monitor.

```
int potPin = A0;
int LEDpin = 13;
int val=0;
void setup()
{
  pinMode(A0, INPUT);
  pinMode(LEDpin, OUTPUT);
  Serial.begin(9600);
}
void loop()
{
  val = analogRead(A0); // read the value at A0
  Serial.println(val);
  digitalWrite(LEDpin, HIGH); // turn the LED on
  delay(val); // Wait for val millisecond(s)
  // turn the LED off
  digitalWrite(LEDpin, LOW);
  delay(val); // Wait for val millisecond(s)
}
```

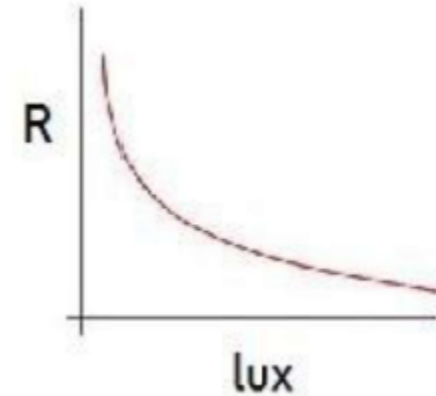


LDR

LIGHT DEPENDENT RESISTOR (LDR)

LDR, or **photoresistor**, is an analog sensor used to quantify incident light as the resistance of the LDR decreases with increasing incident light.

Intensity of incident light $\nearrow \rightarrow$ Resistance $\searrow$



LDR is used to sense the presence and absence of light.

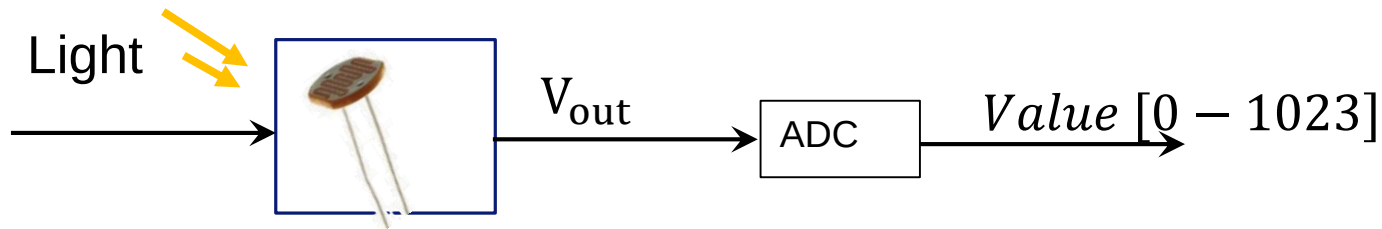
Application example: automatic turning ON/OFF of street lights

LIGHT DEPENDENT RESISTOR (LDR)

LDR is combined with a 4.7kΩ resistor to form a voltage divider

- $V_{out} = V_{in} \left(\frac{R_{resistor}}{R_{resistor} + R_{LDR}} \right)$

Incident light $\rightarrow R_{LDR} \rightarrow V_{out}$

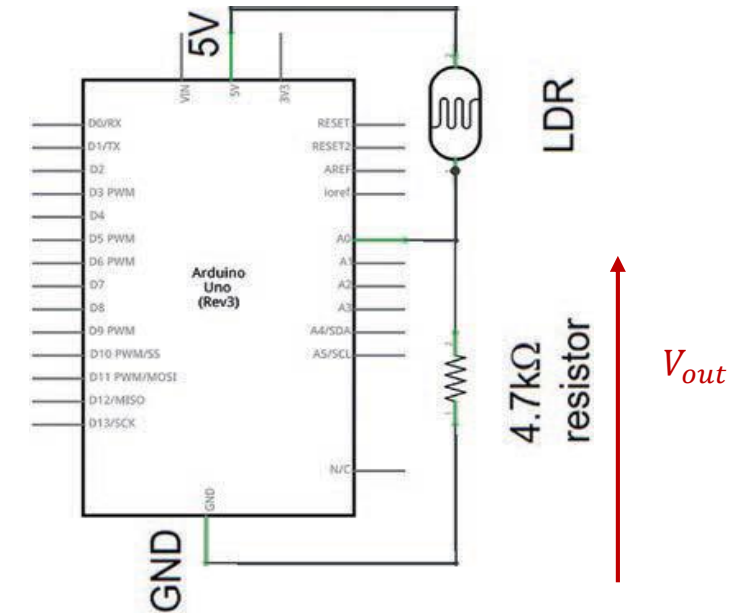


Arduino reads V_{out} using `reading = analogRead(analogPin);`

Recall: Reading value is between 0(0V) and 1023 (+5V)

- $V_{in} \rightarrow 1023$
- $V_{out} \rightarrow \text{reading value}$

$$\rightarrow \text{reading value} = \frac{V_{out} \times 1023}{V_{in}}$$



$$R_{LDR} = \left(\frac{1023}{\text{reading}} - 1 \right) R_{resistor}$$

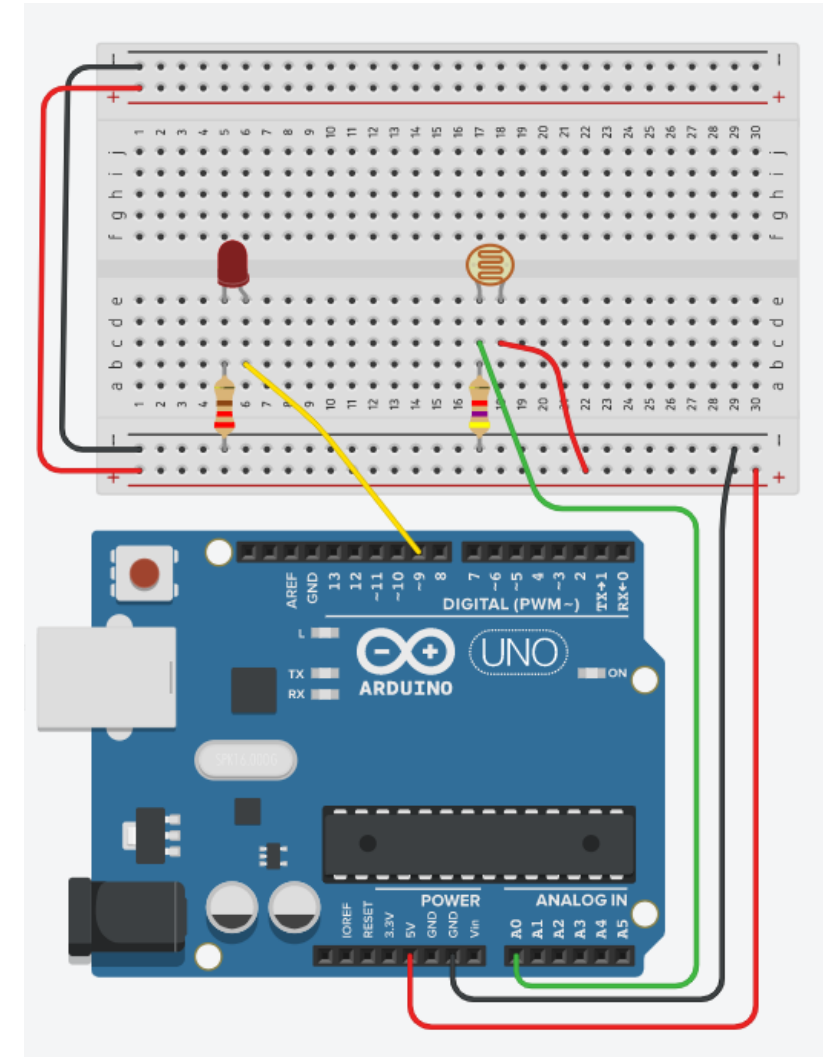
$R_{LDR} = 700\Omega$ in light conditions, reading = 890
 $R_{LDR} = 50k\Omega$ in dark, reading = 90

LIGHT DEPENDENT RESISTOR (LDR) – EXAMPLE 1

Set LED to a relative brightness based on how much light the sensor detects

- The higher the light intensity, the lower the LED brightness
- The LED is connected to the PWM pin 9

4.7k Ω may not be available in the kit.
Instead, use two 10k Ω resistors in parallel.

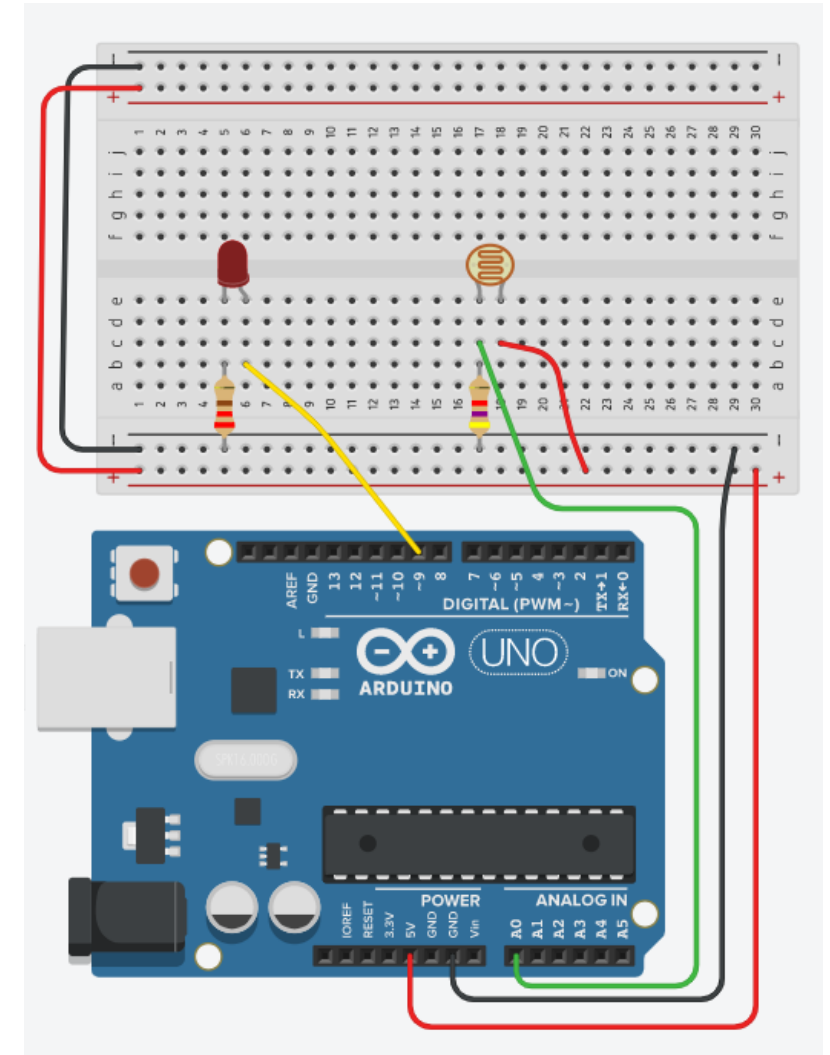


LIGHT DEPENDENT RESISTOR (LDR) – EXAMPLE 1

```
int ldrPin = A0;
int ldrReading = 0;
int ledPin = 9;
int brightness = 0;

void setup()
{
  pinMode(ldrPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop()
{
  // read the value from the sensor
  ldrReading = analogRead(ldrPin);
  // map the sensor reading to a range for the LED
  brightness = map(ldrReading, 1023, 0, 0, 255);
  // change LED brightness
  analogWrite(ledPin, brightness);
  delay(100); // Wait for 100 millisecond(s)
}
```



LIGHT DEPENDENT RESISTOR (LDR) – EXAMPLE 2

Use the LDR to change the brightness of an LED, depending on the incident light such as night light:

When $\text{ldrReading} < 500$ (low light intensity):

The LED is turned on

The LED brightness depends on sensorValue (using PWM):

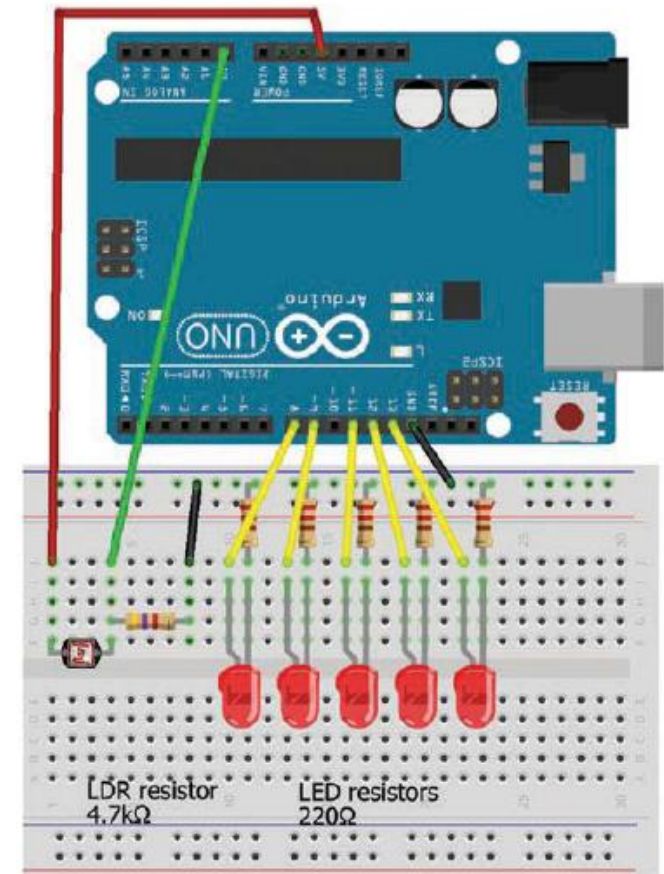
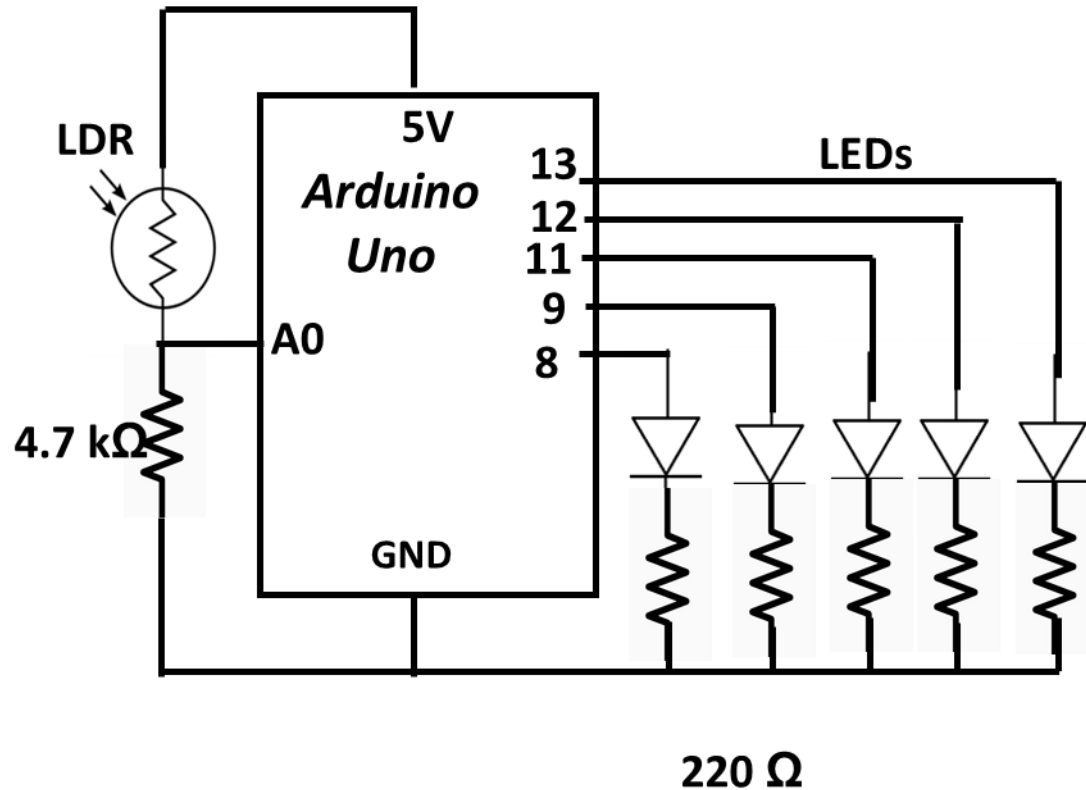
The higher the value of sensorValue the lower the brightness

Otherwise ($\text{ldrReading} \geq 500$):

The LED is turned off

LIGHT DEPENDENT RESISTOR (LDR) – EXAMPLE 3

Use an LDR to turn on a number of LEDs, a brighter light turning on more LEDs



LIGHT DEPENDENT RESISTOR (LDR) – EXAMPLE 3

```
int Vdivid = A0;    // voltage divider analog pin
int nLEDs = 5;      // number of LEDs
int LEDpin[] = {8, 9, 11 ,12, 13}; // LED pins
int reading, level;
void setup()
{ // define LED pins as outputs
  for (int i=0; i<5; i++)
    pinMode(LEDpin[i], OUTPUT);
}
void loop()
{
  reading = analogRead(Vdivid); // voltage divider reading
  level = 0;                    // set number of LEDs to zero
  level = map(reading, 0, 1023, 0, nLEDs+1); // map reading to level
  for (int i = 0; i < nLEDs; i++)
  { // turn on LED
    if (i < level)
      digitalWrite(LEDpin[i], HIGH); // less than level
    else
      digitalWrite(LEDpin[i], LOW);  // otherwise turn off LED
  }
  delay(1000); // delay 1000ms
}
```

TEMPERATURE SENSOR

TEMPERATURE SENSOR : LM35DZ

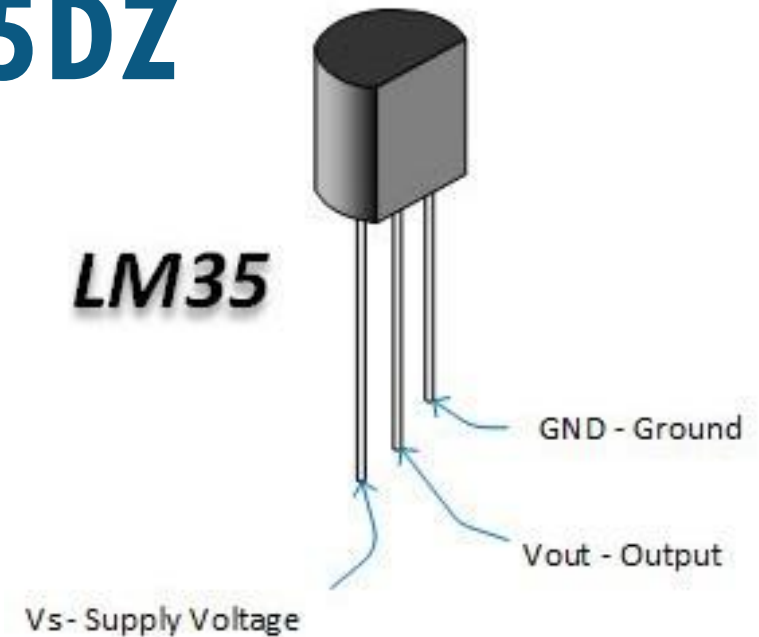
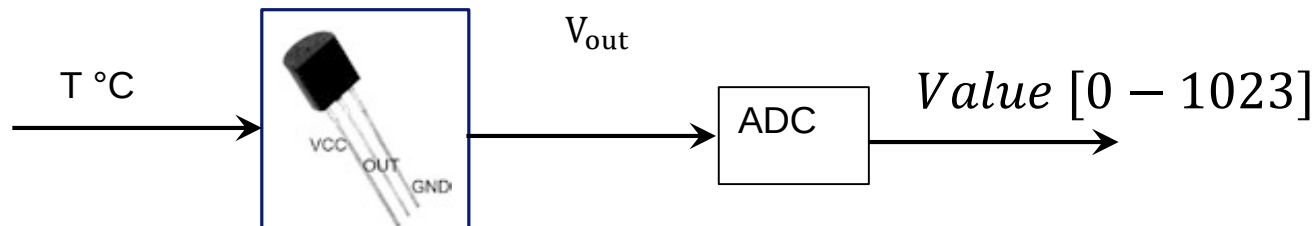
- The LM35DZ is a precision temperature analog sensor:
 - Operating range: 0° C to 100° C
 - Vout increases by 10mV for 1° C increase in temperature
 - Vout range : 0V to 1V (1V = 100°C × 10mV)

To convert the voltage to temperature :
Temp in °C = (Vout in V) * 100

Example: $T = 20^{\circ}\text{C} \rightarrow V_{\text{out}} = 20 \times 10 = 200\text{mV} = 0.2 \text{ V}$

Maximum output voltage $V_{\text{max}} = 100 \times 10 \text{ mV} = 1 \text{ V}$

Arduino analog-to-digital conversion (ADC) converts a voltage to a digital value between 0 and 1023.



TEMPERATURE SENSOR : LM35DZ

analogReference()

analogReference configures the reference voltage.

It takes one parameter:

DEFAULT: the default analog reference of 5 volts

INTERNAL: a built-in reference, equal to 1.1 volts

EXTERNAL: the voltage applied to the AREF pin (0 to 5V only) is used as the reference.

The Arduino has three analog reference values:

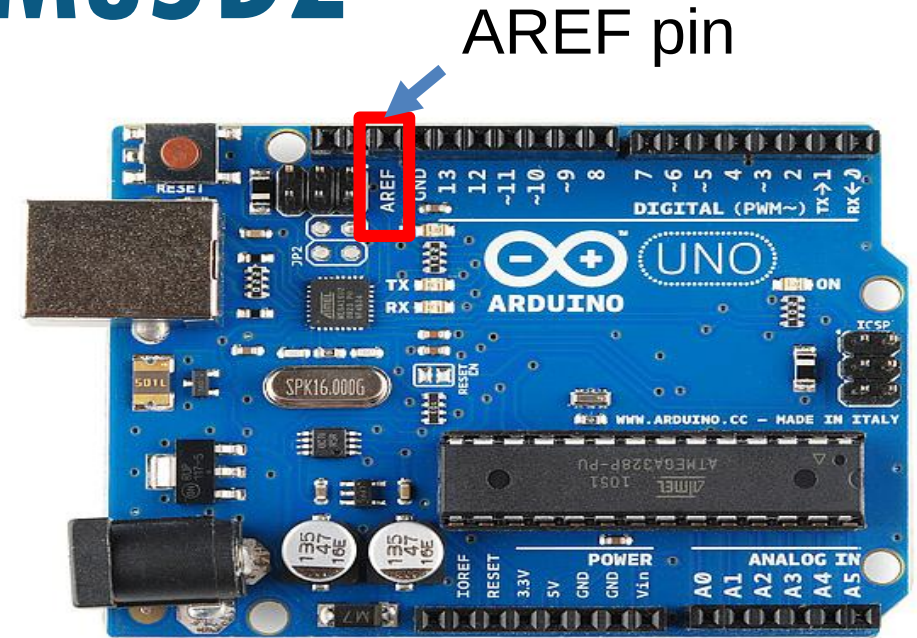
analogReference(DEFAULT): equates 5V to 1023

analogReference(INTERNAL): equates 1.1V to 1023

analogReference(EXTERNAL): equates 3.3V to 1023 when the 3.3V pin is connected to AREF pin

Example : For the code below, the value of reading will be equal to 1023 if the input voltage is equal to 1.1 volts:

```
analogReference(INTERNAL);  
reading = analogRead(A0);
```



TEMPERATURE SENSOR : LM35DZ

Default ADC maximum voltage $V_{\max_default}^{ADC} = 5V$

- $5V \rightarrow 1023$
- $1V \rightarrow ? = 1023/5 = 205$
- Range of output $[0-1V] \rightarrow [0-205]$

Changing the ADC maximum voltage to 1.1 V

- $1.1 V \rightarrow 1023$
 - $1V \rightarrow ? = 1023/1.1 = 930$
 - Range of output $[0-1V] \rightarrow [0-930]$
- ✓ provide greater resolution for the temperature sensor.

Example: temperature increase of $1^\circ C$ corresponds to

- For $V_{\max_default}^{ADC} = 5V$: $1^\circ C \rightarrow 10 \text{ mV} \rightarrow 1023 \cdot 0.01/5 = 2$ in ADC range
- For $V_{\max}^{ADC} = 1.1 V$: $1^\circ C \rightarrow 10 \text{ mV} \rightarrow 1023 \cdot 0.01/1.1 = 9$ in ADC range

TEMPERATURE SENSOR : LM35DZ

Connection To Arduino and visualise the data on serial monitor/plotter.

- output pin → One of Arduino analog input pin (A0- A5)

```
int tempPin = A0; // define LM35DZ signal on analog pin A0
int reading;      // define reading as an integer
float temp;       // define temp as a real number
void setup()
{
  Serial.begin(9600); // define Serial output baud rate
  analogReference(INTERNAL); // set ADC voltage to 1.1V rather than 5V
}
void loop()
{
  reading = analogRead(tempPin); // read temperature sensor pin
  temp = (reading * 110.0)/1023; // convert reading to temperature
  Serial.println(temp);
  delay(200);
}
```

TEMPERATURE SENSOR : TMP36

Tinkercad does not include LM35DZ, it has another sensor called **TMP36**

- It is a precision temperature sensor:
- Operating range: -40°C to $+125^{\circ}\text{C}$
- Vout range : 0V to 1.75V
- Vout increases by 10mV for 1°C increase in temperature

```
int reading = 0;  
float vout = 0;  
reading = analogRead(Analogpin);
```

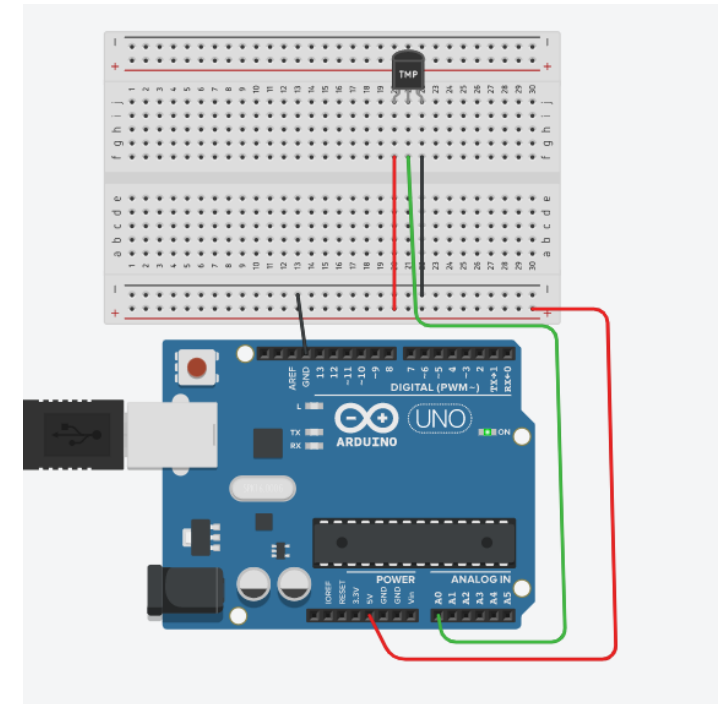
Analog-in pins A0 to A5 can interpret voltages between 0 and 5V, and translate that voltage to a value between 0 and 1023

To convert *reading* to the voltage value, we apply the cross-product formula:

$1023 \rightarrow 5 \text{ volts}$

$\text{reading} \rightarrow \text{voltage volts}$

$\text{voltage} = \text{reading} * 5.0 / 1023;$



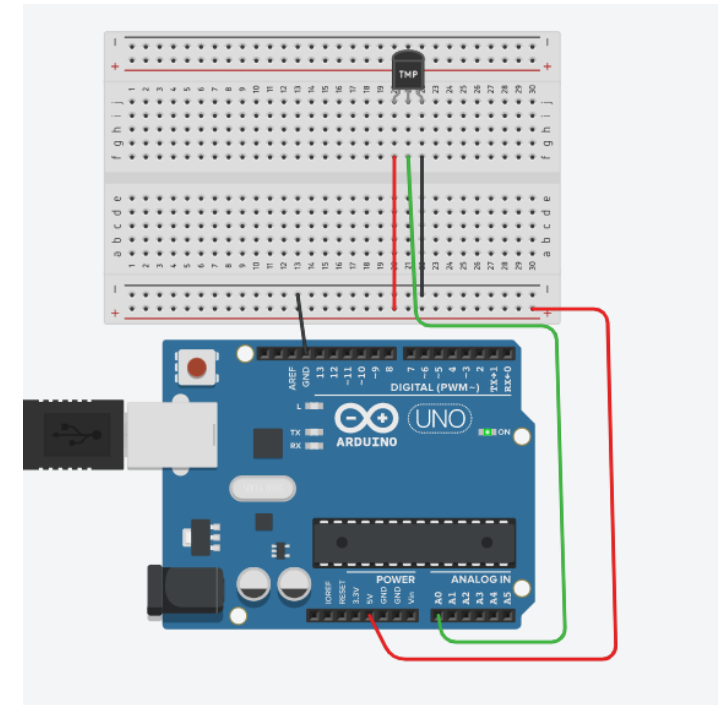
To convert the voltage to temperature, simply use the basic formula:

$$\text{Temp in } ^{\circ}\text{C} = [(\text{Vout in V}) - 0.5] * 100$$

If the voltage out is 1V that means that the temperature is $((1 \text{ V} - 0.5) * 100 = 50^{\circ}\text{C})$

TEMPERATURE SENSOR : EXERCISE 1

```
int analogpin = A0;
int reading= 0;
float temp = 0;
float vout = 0;
void setup()
{
  pinMode(analogpin, INPUT);
  Serial.begin(9600);
}
void loop()
{
  reading = analogRead(analogpin);
  //convert the reading value to a voltage between 0V and 5V
  vout = reading*5.0/1023;
  //convert the voltage to temperature
  temp = (vout-0.5)*100;
  Serial.print("Vout: ");
  Serial.println(vout);
  Serial.print("Reading value: ");
  Serial.println(reading);
  Serial.print("Temperature: ");
  Serial.println(temp);
  delay(100);
}
```



TEMPERATURE SENSOR : EXERCISE 2

Circuit behavior:

Temperature < 10 :

- Turn off all LEDs

Temperature between 10 and 20:

- Turn on one LED

Temperature between 20 and 30:

- Turn on two LEDs

Temperature ≥ 30 :

- Turn on all LEDs

